

ATIVIDADE PRÁTICA: CAMADA DE APLICAÇÃO E TRANSPORTE

Disciplina: Redes de Computadores

Confirmação de Entrega e Reenvio Manual em Nível de Aplicação

A atividade foi implementada utilizando sockets UDP (SOCK_DGRAM), conforme solicitado. Como o UDP não garante a entrega dos datagramas, foi criado um mecanismo de confirmação e reenvio na camada de aplicação.

Cada mensagem enviada pelo cliente recebe um identificador numérico sequencial e utiliza o formato:

MSG|<ID>|<CONTEUDO>

Antes do envio, a mensagem é armazenada no dicionário `pending_messages`. Dessa forma, o cliente mantém o controle das mensagens que ainda não receberam confirmação.

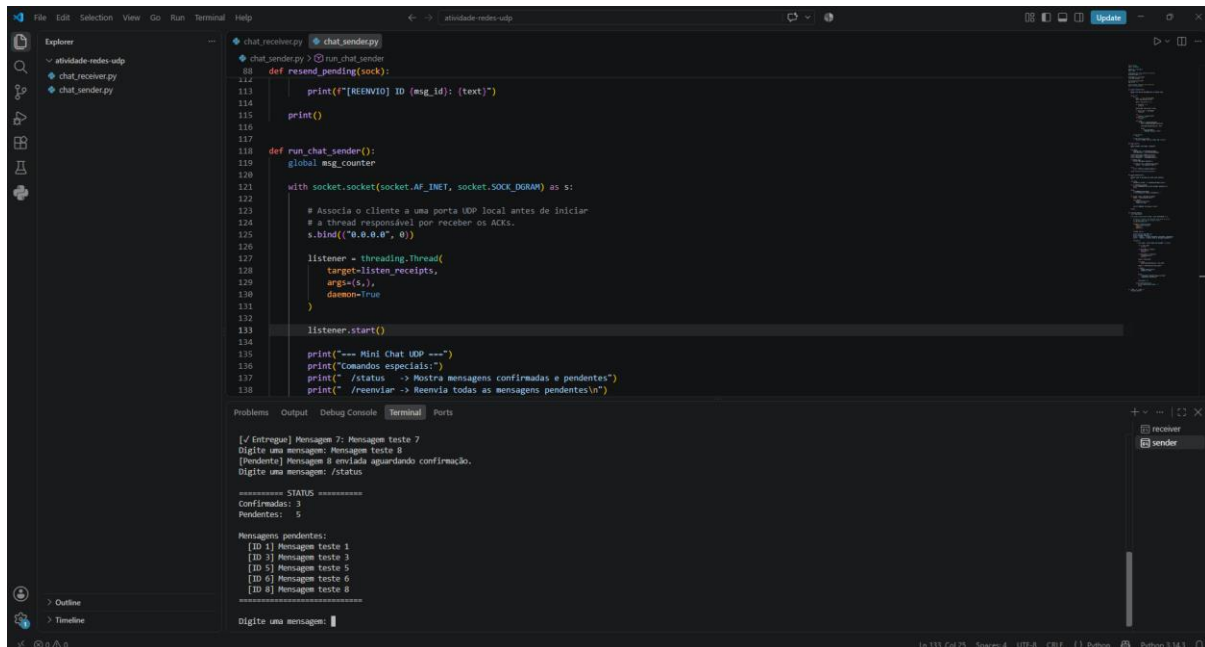
Quando o servidor recebe uma mensagem corretamente, ele envia ao cliente um recibo no formato:

DELIVERED|<ID>

O cliente possui uma thread executada em segundo plano que recebe esses recibos. Quando uma confirmação é recebida, a mensagem correspondente é removida de `pending_messages` e passa a ser considerada entregue.

O servidor também utiliza uma taxa de descarte artificial de 40%, configurada através de `DROP_RATE = 0.4`, simulando perdas de pacotes durante a comunicação.

O comando /status permite visualizar as mensagens confirmadas e as mensagens que continuam pendentes. O comando /reenviar percorre as mensagens ainda pendentes e realiza uma nova transmissão utilizando UDP.



The screenshot shows a VS Code editor with a Python file named `chat_receiver.py`. The code defines a `run_chat_sender` function that uses a `socket` to connect to a server and a `threading.Thread` to listen for incoming messages. The terminal output shows the client running and displaying the status of messages.

```
def run_chat_sender():
    global msg_counter
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        # Associa o cliente a uma porta UDP local antes de iniciar
        # e a thread responsável por receber os ACKs.
        s.bind(("0.0.0.0", 0))

        listener = threading.Thread(
            target=listen_receipts,
            args=(s,),
            daemon=True
        )
        listener.start()

    print("=== Mini Chat UDP ===")
    print("Comandos especiais:")
    print(" /status -> Mostra mensagens confirmadas e pendentes")
    print(" /reenviar -> Reenvia todas as mensagens pendentes\n")
```

Terminal Output:

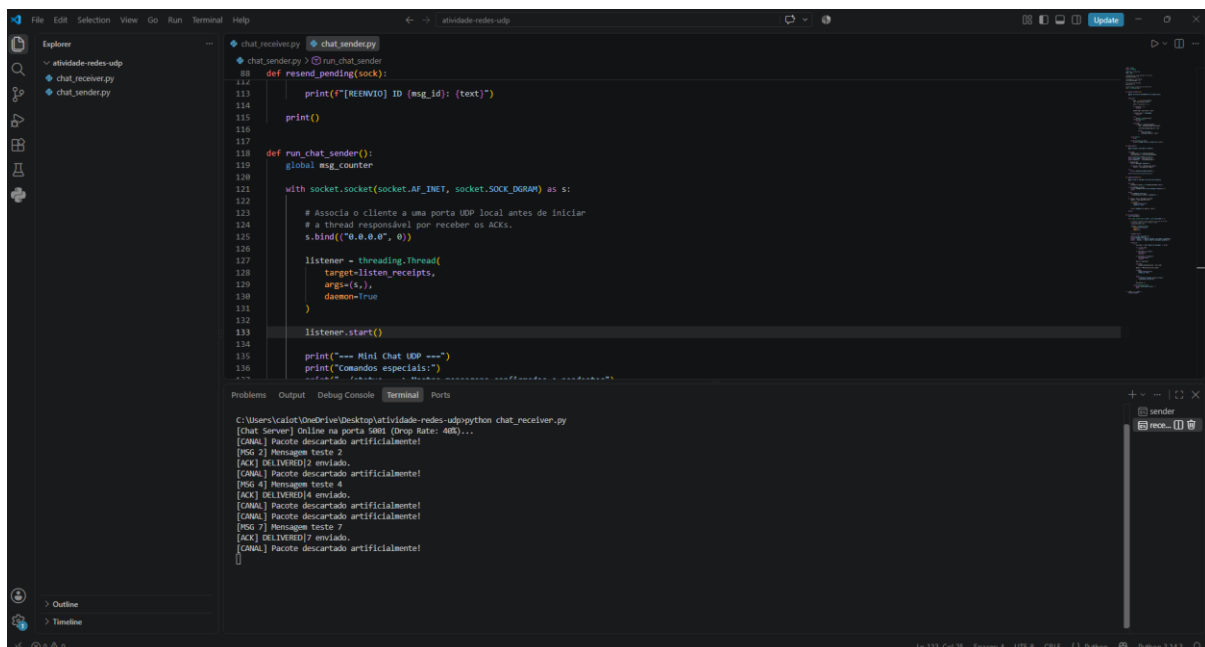
```
[/ Entregue] Mensagem 7: Mensagem teste 7
Digite uma mensagem: Mensagem teste 8
[Pendente] Mensagem 8 enviada aguardando confirmação.
Digite uma mensagem: /status

===== STATUS =====
Confirmadas: 3
Pendentes: 5

Mensagens pendentes:
[ID 1] Mensagem teste 1
[ID 3] Mensagem teste 3
[ID 5] Mensagem teste 5
[ID 6] Mensagem teste 6
[ID 8] Mensagem teste 8

Digite uma mensagem:
```

Figura 1A – Cliente exibindo, através do comando /status, as mensagens confirmadas e as mensagens que permaneceram pendentes.



The screenshot shows a VS Code editor with a Python file named `chat_receiver.py`. The code defines a `run_chat_sender` function that uses a `socket` to connect to a server and a `threading.Thread` to listen for incoming messages. The terminal output shows the server running and displaying the status of messages.

```
def run_chat_sender():
    global msg_counter
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        # Associa o cliente a uma porta UDP local antes de iniciar
        # e a thread responsável por receber os ACKs.
        s.bind(("0.0.0.0", 0))

        listener = threading.Thread(
            target=listen_receipts,
            args=(s,),
            daemon=True
        )
        listener.start()

    print("=== Mini Chat UDP ===")
    print("Comandos especiais:")
    print(" /status -> Mostra mensagens confirmadas e pendentes")
    print(" /reenviar -> Reenvia todas as mensagens pendentes\n")
```

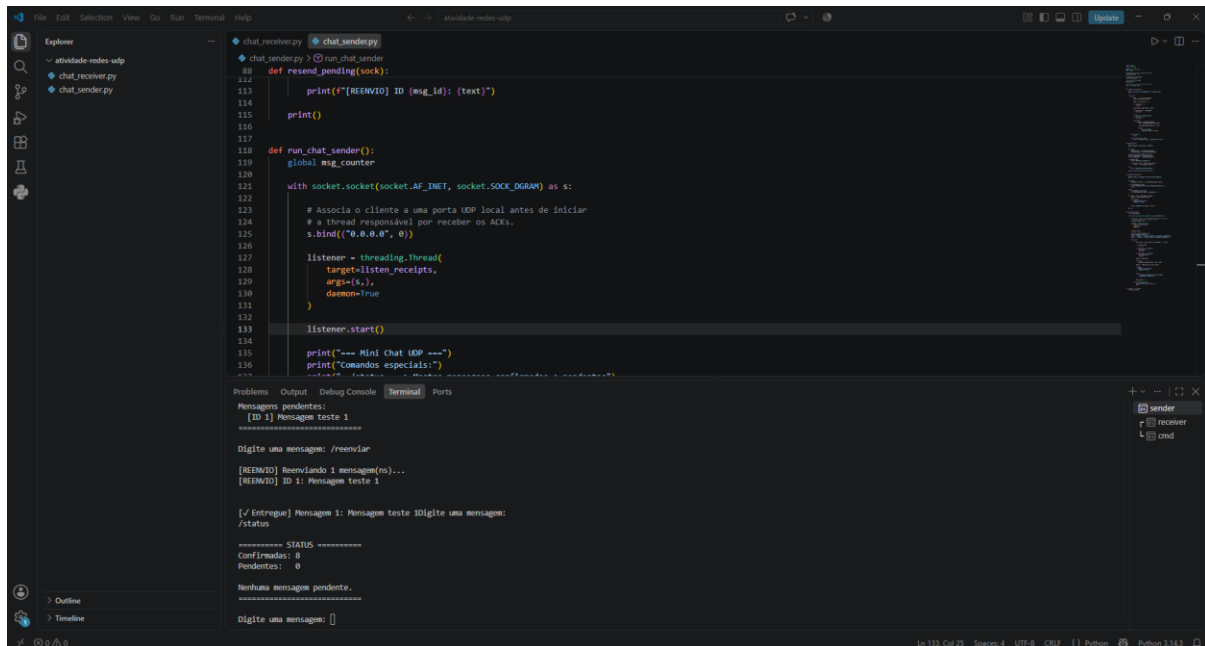
Terminal Output:

```
C:\Users\caio\OneDrive\Desktop\atividade-redes-udp\python chat_receiver.py
[Chat Server] Online na porta 5001 (Drop Rate: 40%)...
[CANAL] Pacote descartado artificialmente!
[MSG 2] Mensagem teste 2
[ACK] DELIVERED! enviado.
[CANAL] Pacote descartado artificialmente!
[MSG 4] Mensagem teste 4
[ACK] DELIVERED! enviado.
[CANAL] Pacote descartado artificialmente!
[MSG 7] Mensagem teste 7
[ACK] DELIVERED! enviado.
[CANAL] Pacote descartado artificialmente!
```

Figura 1B – Servidor demonstrando o descarte artificial de datagramas durante a comunicação UDP.

Durante o teste, foram enviadas oito mensagens. Algumas mensagens chegaram corretamente ao servidor e tiveram seus respectivos recibos DELIVERED enviados ao cliente, enquanto outras foram descartadas pela simulação de perda do canal.

As mensagens que não receberam confirmação permaneceram registradas no dicionário `pending_messages`. O comando `/status` permitiu visualizar essa situação, mostrando três mensagens confirmadas e cinco mensagens pendentes naquele momento da execução.



```
def resend_pending(sock):
    print(f"[REENVID] ID {msg_id}: {text}")
    print()

def run_chat_sender():
    global msg_counter

    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        # Associa o cliente a uma porta UDP local antes de iniciar
        # a thread responsável por receber os ACKs.
        s.bind(("0.0.0.0", 0))

        listener = threading.Thread(
            target=listen_receipts,
            args=(s,),
            daemon=True
        )

        listener.start()

    print("=== Mini Chat UDP ===")
    print("Comandos especiais:")

# Mensagens pendentes:
# [ID 1] Mensagem teste 1
# =====
# Digite uma mensagem: /reenviar
# [REENVID] Reenviando 1 mensagem(n)
# [REENVID] ID 1: Mensagem teste 1
#
# [✓ Entrega] Mensagem 1: Mensagem teste 1
# Digite uma mensagem: /status
# ===== STATUS =====
# Confirmadas: 3
# Pendentes: 5
#
# Nenhuma mensagem pendente.
# =====
# Digite uma mensagem: []
```

Figura 2 – Reenvio de mensagem pendente, recebimento da confirmação e estado final sem mensagens pendentes.

Para recuperar as mensagens que não haviam sido confirmadas, foi utilizado o comando `/reenviar`. O cliente percorreu o conjunto de mensagens existentes em `pending_messages` e realizou novamente o envio utilizando o mesmo identificador de cada mensagem.

Como a perda é aleatória, o comando pode ser utilizado novamente caso algum reenvio também seja descartado. Ao final do teste, todas as oito mensagens receberam confirmação de entrega. O comando `/status` apresentou oito mensagens confirmadas e zero mensagens pendentes.

Na implementação desenvolvida, a confiabilidade da comunicação é controlada manualmente pela camada de aplicação. A própria aplicação precisa identificar as mensagens, armazenar aquelas que ainda não foram confirmadas, gerar recibos de entrega e realizar retransmissões quando necessário.

O UDP é um protocolo de transporte orientado a datagramas e não garante, por si só, que os dados chegarão ao destino. Por esse motivo, a ausência do recibo DELIVERED é utilizada nesta atividade para identificar uma mensagem ainda pendente.

No TCP, mecanismos de confiabilidade são fornecidos pelo próprio protocolo de transporte. O TCP utiliza recursos como números de sequência, confirmações e retransmissões para fornecer à aplicação um fluxo de bytes confiável e ordenado.

Portanto, nesta atividade, funções normalmente associadas à confiabilidade foram implementadas diretamente na camada de aplicação sobre UDP, permitindo observar de forma prática a diferença entre os dois protocolos de transporte.

A atividade permitiu implementar e testar um mecanismo simples de confiabilidade sobre UDP. Através do uso de identificadores, mensagens pendentes, recibos DELIVERED e reenvio manual, foi possível detectar perdas simuladas e realizar novas transmissões até que todas as mensagens fossem confirmadas.

O experimento demonstrou que, ao utilizar UDP, a aplicação precisa implementar seus próprios mecanismos caso necessite de confirmação de entrega. Esse comportamento contrasta com o TCP, no qual a camada de transporte já oferece mecanismos de confiabilidade.